

INFORME TÉCNICO

Cambios y decisiones técnicas del frontend

TDDLab / WhiteBox · Arquitectura de presentación frontend

Proyecto	TDDLab / WhiteBox
Responsable	Andrés Salvatierra Ramírez (PO)
Fecha	3 de mayo de 2026
Tipo	Documentación técnica de frontend

Síntesis ejecutiva

El frontend fue reorganizado con una arquitectura de presentación por dominios, orientada a reducir acoplamiento, unificar criterios visuales y dejar una base escalable para las pantallas principales del sistema. La decisión de usar presentation/ como capa principal permitió expresar con claridad qué parte del código pertenece a la experiencia de usuario y qué parte corresponde a lógica de negocio, datos o integración.

🎯 1. Objetivo del informe

Documentar los cambios realizados en el frontend y justificar la decisión arquitectónica adoptada durante los tres épics de rediseño y refactor. El informe explica cómo se estructuró la capa de presentación, qué criterios técnicos guiaron el cambio y qué beneficios aporta esta dirección para la mantenibilidad del sistema.

🔭 2. Lectura general del cambio

La intervención no se limitó a mejorar la apariencia de pantallas aisladas. El trabajo se abordó como una reorganización progresiva del frontend, donde cada pantalla priorizada pasó a responder a una estructura más clara: composición de página, componentes visuales, hooks de comportamiento, servicios o adapters y tipos propios del dominio.

Con esta decisión, la antigua dependencia de una capa genérica de secciones dejó de ser el eje de la presentación. En su lugar, la responsabilidad visual y de interacción quedó concentrada en `presentation/`, mientras `shared/` reúne los elementos reutilizables que dan coherencia al sistema.

3. Contexto y motivación

- Se redujo la lógica embebida en vistas grandes, separando composición visual, comportamiento de pantalla y acceso a datos.
- Se corrigió la dispersión de criterios visuales entre pestañas, tablas, formularios, estados de pantalla y acciones comunes.
- Se reorganizó la presentación del sistema alrededor de dominios funcionales, evitando que las pantallas dependan de una carpeta genérica para resolver responsabilidades distintas.
- Se fortaleció la reutilización de componentes compartidos para disminuir duplicación técnica y mejorar la coherencia entre flujos.
- Se preservaron rutas y flujos principales durante la migración, manteniendo compatibilidad funcional mientras se ordenaba la base del frontend.

4. Decisión arquitectónica

La decisión central fue adoptar una arquitectura de presentación por dominios, inspirada en el enfoque *feature-first*. A nivel conceptual, el código se organiza alrededor de capacidades funcionales del producto; a nivel físico, por recomendación del magíster, la carpeta principal se denomina `presentation/` para comunicar que allí vive la capa visual y de interacción del frontend.

Este nombre evita ambigüedad: no todo lo que pertenece a una *feature* debe vivir necesariamente en la presentación. La lógica de negocio, casos de uso o integraciones pueden permanecer en módulos especializados, mientras `presentation/` agrupa las piezas necesarias para construir la experiencia de usuario de cada pantalla.

Capa	Responsabilidad
<code>presentation/</code>	Capa principal de presentación. Agrupa las pantallas por dominio funcional y contiene páginas, componentes, hooks, servicios/adapters y tipos propios.
<code>shared/</code>	Base común para layouts, estados de pantalla, diálogos, botones, feedback, helpers y patrones reutilizables entre pantallas.
<code>modules/</code>	Capa que conserva casos de uso, puertos, repositorios o integraciones existentes cuando corresponden al dominio de negocio o infraestructura.
<code>app/</code>	Punto de integración de rutas, proveedores y configuración global del frontend.

🎯 5. Estructura objetivo

```
src/
├── app/
├── shared/
├── presentation/
│   └── <pantalla-o-dominio>/
│       ├── pages/
│       ├── components/
│       ├── hooks/
│       ├── services/
│       └── types/
└── modules/
```

🧩 6. Patrones aplicados

- Container / Presentational para separar composición de pantalla y piezas visuales reutilizables.
- Custom hooks para concentrar estado, navegación, filtros, selección y comportamiento de cada pantalla.
- Servicios o adapters para aislar acceso a datos, repositorios existentes e integración con módulos previos.
- Componentes compartidos para headers, layouts, estados de carga, empty states, errores, acciones, diálogos y feedback.
- Fuente de verdad clara para filtros, selección y contexto de pantalla, evitando duplicar estado entre URL, estado local y almacenamiento persistente.
- Renderizado por rol para mostrar acciones disponibles según permisos sin mezclar reglas de acceso dentro de componentes puramente visuales.

Lineamientos técnicos

- Evitar alert, confirm y window.location.reload, reemplazándolos por feedback y diálogos controlados por componentes.
- Reducir estilos inline y favorecer estilos centralizados, clases reutilizables o componentes compartidos.
- Mantener estados de pantalla consistentes: loading, empty, error y success.
- Preservar rutas y flujos principales durante la migración arquitectónica.
- Priorizar componentes pequeños, responsabilidades claras y contratos de datos explícitos.

7. Impacto por pantalla

Área	Impacto técnico
Homepage	Se estableció como entrada autenticada del sistema, alineada con la navegación principal y con los patrones visuales comunes.
Landing pública	Se incorporó como entrada pública del producto, conectando la presentación inicial con el flujo de autenticación sin acoplarla al resto de pantallas internas.
Autenticación	Se rediseñó como pantalla independiente, coherente con la identidad visual del sistema y separada de la antigua capa de presentación.
Grupos	Se reorganizó la gestión de grupos con jerarquía visual clara, acciones consistentes, navegación hacia participantes y separación entre UI, hooks, servicios y tipos.
Tareas	Se alinearon listados, filtros, ordenamiento, acciones y navegación bajo una estructura más mantenible y compatible con los patrones compartidos.
Detalle de tarea	Se ordenó la presentación de información, entregas, acciones y estados para reducir lógica acoplada en la vista.
Mis prácticas	Se consolidaron patrones de lista, formularios, diálogos y feedback para mantener consistencia con el resto del frontend.
Detalle de práctica	Se alineó con la arquitectura de presentación, separando información, acciones, estado de pantalla y navegación.
Usuarios	Se estructuraron búsqueda, filtros, tablas, acciones y diálogos bajo componentes más claros y reutilizables.
Ajustes	Se ordenó la configuración del sistema mediante componentes dedicados, separación de comportamiento y presentación coherente con las demás pestañas.
Pantallas complementarias	Los flujos de invitaciones, visualización TDD y Asistente IA quedaron considerados dentro del mismo criterio de presentación y consistencia visual.

8. Beneficios técnicos

- Menor acoplamiento entre pantallas, navegación, estado y acceso a datos.
- Mayor coherencia visual entre pestañas, formularios, tablas, diálogos, feedback y estados de pantalla.
- Código más fácil de leer porque cada dominio funcional concentra sus piezas relacionadas en un mismo lugar.

- Mejor reutilización de componentes compartidos sin duplicar soluciones por pantalla.
- Base más escalable para incorporar nuevas pantallas o funcionalidades sin romper la coherencia del sistema.
- Separación más clara entre presentación, comportamiento y servicios/adapters.
- Mejor trazabilidad para QA, mantenimiento y evolución del frontend.

9. Cierre arquitectónico

Conclusión técnica

La arquitectura adoptada mejora la lectura del código y la consistencia del producto porque separa responsabilidades donde antes convivían en las mismas vistas. presentation/ funciona como una capa explícita para la experiencia de usuario; shared/ evita duplicación transversal; y modules/ conserva las piezas de dominio o integración que no deberían contaminar la UI. Esta separación permite evolucionar el frontend sin que cada nueva pantalla incremente el acoplamiento del sistema.

10. Bibliografía y recursos

Chohra, M. (2025). The Guide to scale your React and React Native app: Feature First Architecture. JavaScript in Plain English. Consultado el 3 de mayo de 2026.

<https://javascript.plainenglish.io/the-guide-to-scale-your-react-and-react-native-app-feature-first-architecture-44f1eef72976?gi=9c426f99518b>

Recursos del proyecto

Recurso	Plataforma	Enlace
Tablero de gestión	Trello	https://trello.com/b/StcYGS06/white-box
Repositorio del proyecto	GitHub	https://github.com/UCB-TallerDeDesarrollo/TDDLab/tree/WhiteBox
Diseño de interfaz	Figma	https://www.figma.com/design/FZ48aPFT2PXOWcklywMPCf/WHITE-BOX-TTD-LAB?node-id=171-565&p=f&t=zU7jQSwahHFvMevc-0